
PROYECTO FINAL

RenovaCalzado



Fundamentos de programación

25 DE SEPTIEBRE DE 2026
DAYANA LIZETH ROBLES MORENO
Matrícula: 07320081

I. Análisis Organizacional.

Datos generales.

Empresa: “RenovaCalzado”

Dirección: C. Río Usumacinta 23, El Pocito, Colonia, 76190 San José de los Olvera, Qro

RenovaCalzado es un negocio dedicado a la renovación de calzado. Su misión es extender la vida útil de los zapatos y ofrecer servicios a costos accesibles.

El área en la que se busca mejorar es el registro de pedidos, enfocándonos en la cantidad de servicios que puede recibir, el cálculo de costos contemplando descuentos y recargos por servicio Express, control de tiempos de entrega, y, por último, los anticipos y saldos pendientes.

Definición del problema.

En la empresa llevan el registro de manera manual, escribiendo en cinta canela únicamente el nombre del cliente y el servicio. Esto puede generar pérdida de información y errores en el cálculo de costos, descuentos, anticipos y saldos pendientes, además de confusiones con los tiempos de entrega. Por ello, se busca desarrollar un algoritmo que facilite el registro y cálculo de montos para llevar un mejor control de los pagos realizados por el cliente.

Reglas del negocio:

Servicio	Precio	Límite c. servicio	Tiempo Inicial	Par extra
Cambio de suela	\$100	3	3 días	+2 días
Costuras	\$300	3	5 días	+3 días
Tintado	\$500	2	7 días	+4 días
Restauración de gamuza	\$900	2	9 días	+5 días
Ajuste de correa	\$20	9	0 días (hasta 2 pares) / 1 día (3 a 5 pares)	+1 día (a partir del 6º par)

1. Solo puede ingresar hasta 10 servicios diferentes por ticket.
2. Cada servicio tiene un límite de cantidad; Cambio de suela: 3, Costuras: 3, Tintado: 2, Restauración Gamuza: 2, Ajuste Correa: 9.
3. Si su pedido es de más de 2 servicios, se aplica un 10% de descuento sobre el subtotal.

4. Los pedidos 'Express' reciben un recargo del 20% sobre el subtotal y se reduce el tiempo de entrega a la mitad.
5. Para iniciar con el servicio el cliente puede liquidar o dejar un anticipo de mínimo el 50%
6. El tiempo de entrega depende de la cantidad de pares y del tipo de servicio. Si se solicitan varios servicios, se toma el que tenga mayor tiempo de entrega.
7. No se puede modificar el archivo de catalogo_servicios.txt solo es archivo lectura.

II. ESTRUCTURA DEL PROGRAMA.

Diccionarios Principales.

- servicios: Diccionario que almacena los nombres de los 5 servicios disponibles y sus costos base.
- dic_tiempo_entrega: Diccionario que guarda los valores estructurados como [tiempo_base, tiempo_extra, limite_maximo] para cada servicio.
- archivos_disponibles: Diccionario que relaciona una clave numérica (1 al 5) con los archivos de texto del sistema (catalogo_servicios.txt, politicas_del_negocio.txt, promociones_Especiales.txt, bitacora_pedidos.txt, Base_datos_clientes.txt).

Operadores y estructuras de control

Operador	Símbolos	Ejemplo	Explicación en el programa
Asignación y acumulación	= +=	subtotal += servicios.get(servicio_real)	Asigna valores iniciales y acumula cantidades de dinero o contadores.
Aritméticos básicos	+ - *	total = subtotal - m_descuento + m_express	Calcula subtotal, montos de descuento, recargos y saldos.
División entera	//	t_entrega = t_entrega // 2	Reduce el tiempo de entrega a días enteros para el servicio Express.
Comparación o relacionales	>= <= > ==	if eleccion_cliente >= minimo and eleccion_cliente <= maximo:	Comprueba los limites por servicio. Compara los tiempos de entrega para determinar el mayor. Valida los datos de entrada
Operadores lógicos	and or	if urgente == "si" or urgente == "no":	Evalúa condiciones simultaneas en validaciones

Estructura		Ejemplo	Explicación en el programa
Estructura iterativa	for i in range()	for pedido in range(1, c_pedidos + 1):	Recorre una secuencia definida para registrar cada servicio pedido.
Definición de funciones	def nombre_funcion(parametros):	def calcular_tiempo_entrega(servicio, t_entrega, cantidad):	Permite modularizar el código en bloques reutilizables que realizan tareas específicas (como calcular tiempos, mostrar menús o validar entradas), recibiendo parámetros de entrada y devolviendo datos mediante return.
Ciclo indeterminado	while True:	while True: anticipo = float(input("Ingrese el anticipo")) if anticipo >= anti_min and anticipo <= total: break	Repite el ciclo hasta que deje de ser válido. En el programa se aplica para repetir el ciclo de la lectura de los datos de entrada hasta que el usuario ingrese un valor válido o dentro del rango.
Condicional múltiple	if elif else	if opcion_menu == 1: ... elif opcion_menu == 2:	Selecciona la ruta de cálculo adecuada según el tipo de servicio elegido.

Nombre	Tipo	Explicación del programa
servicios	dict (diccionario)	Almacena los nombres de los 5 servicios disponibles y sus precios fijos.
dic_tiempo_entrega	dict (diccionario)	Guarda los tiempos base, tiempo extra por par y el límite permitido por cada servicio.
archivos_disponibles	dict (diccionario)	Relaciona un número (1 al 5) con el nombre de cada archivo de texto del sistema.
fecha_formateada_global	str (cadena)	Guarda la fecha ya formateada (00/00/0000) derivada de la tupla para estamparla en los archivos.
ultimo_cliente / ultimo_telefono	str (cadena)	Resguardan los datos del último cliente atendido para poder meterlos automáticamente a la base de datos de clientes si se requiere.
cliente	str (cadena)	Almacena el nombre completo que teclea el usuario al hacer un ticket

c_pedidos	int (entero)	Guarda la cantidad de servicios totales que va a pedir el cliente (del 1 al 10).
ticket_servicios	Lista	Almacena los servicios solicitados para controlar y validar las limitaciones de cantidad por servicio.
Servicio_real	str	Contiene el nombre real del servicio extraído del diccionario según la opción numérica elegida.
subtotal	float (real)	Acumula la suma de los costos de los servicios solicitados
t_entrega	int (entero)	Calcula y compara los días de entrega para quedarse con el mayor de ellos.
m_descuento	float (real)	Guarda el monto del 10% de descuento cuando el pedido pasa de 2 servicios.
m_express	float (real)	Calcula el recargo del 20% si el cliente elige el servicio expres
total	float (real)	Almacena el costo final ya con descuentos y recargos aplicados.
anticipo	float (real)	Registra la cantidad de dinero que deja el cliente (mínimo el 50%)
s_pendiente	float (real)	Calcula lo que le falta por pagar al cliente si no liquidó el total.

III. Requerimientos:

1. **Identificación de Usuario:** La función login(), pide al usuario ingresar su nombre o nickname para identificarlo.
2. **Bienvenida Dinámica:** Usé f-strings (print(f"Bienvenido al sistema, {usuario}.")) para utilizar esa variable de texto y saludar formalmente en la consola de manera personalizada.
3. **Pantalla de Carga:** La función pantalla_carga() usa un ciclo y la librería time.sleep(1) para simular una pausa de 3 segundos con avisos en pantalla antes de mostrar el menú.

4. **Menú como Matriz:** El menú principal lo organicé dentro de `main()` en forma de matriz, esta se recorre con un ciclo para mostrar las opciones.
5. **Control de Inactividad del Usuario:** Dejé programada la función `inactividad()` por si se requiere activar; utiliza un ciclo `for` para medir el tiempo de espera y le exige al usuario escribir exactamente "si" o "no" para continuar o salirse al inicio.
6. **Captura de Fecha Estructurada:** En la función `capturar_fecha()` pido el día, mes y año validando los años bisiestos y los días que tiene cada mes. Y regrese la tupla bajo la sintaxis (dia, mes, anio), la cual luego convierto en una variable global con formato de diagonal para meterla automáticamente en los textos.
7. **Persistencia en Archivos de Texto:** En el diccionario `archivos_disponibles` cuenta con 5 archivos .txt de base (`catalogo_servicios.txt`, `politicas_del_negocio.txt`, `promociones_Especiales.txt`, `registro_clientes.txt`, `bitacora_pedidos.txt`). El programa te deja leerlos completos con `read()`, y para escribir o anexar datos ("w" y "a"), le puse una restricción para que nadie pueda alterar por error el catálogo de servicios, además de darte a escoger si quieres escribirlos desde cero o utilizar los datos del último ticket.
8. **Control de Excepciones del Sistema:** Cree una función llamada `pedir_numero()` que por medio de bloque `try-except` válida las entradas y sus limitaciones. Si alguien escribe una letra en lugar de un número (`ValueError`) o intenta abrir un archivo que no existe (`FileNotFoundError`), el programa no se truea feo de golpe, sino que te mande un mensaje amigable.

9. **Depuración Técnica (Debugging con PDB):** Ocupé el módulo PDB en la consola para corregir errores y ver como se comportaban las variables. También lo combine con prints de control porque había errores que no comprendía donde estaba el problema.

ERROR	DESCRIPCIÓN	SOLUCIÓN
<pre>False 2024 2 28 (28, 2, 2024)</pre> <pre>def capturar_fecha(): bisiesto = False anio = pedir_entero("Ingrese año: ", 2020, 2025) if anio % 4 == 0 and anio % 100 == 0: bisiesto = True mes = pedir_entero("Ingrese mes: ", 1, 12) if mes == 2 and bisiesto: dia = pedir_entero("Ingrese día: ", 1, 29) elif mes == 2 and not bisiesto: dia = pedir_entero("Ingrese día: ", 1, 28) elif mes in [4, 6, 9, 11]: dia = pedir_entero("Ingrese día: ", 1, 30) else: dia = pedir_entero("Ingrese día: ", 1, 31) pdb.set_trace() fecha = (dia, mes, anio) return fecha fecha=capturar_fecha()</pre>	Error: Al probar con el año 2024, la variable booleana de bisiesto se quedaba en False. La condición original fallaba al evaluar la regla de los siglos y el operador and, impidiendo que los años bisiestos cambiaran a True y bloqueando el día 29 en febrero. Solución: Corrección en la estructura condicional por la regla matemática correcta: $\text{if } (\text{anio} \% 4 == 0 \text{ and } \text{anio} \% 100 != 0) \text{ or } (\text{anio} \% 400 == 0)$ Con ayuda del depurador, se comprobó que al ingresar 2024 la variable cambiaba a True, permitiendo capturar el 29 de febrero	<pre>True 29 2 2024 (29, 2, 2024)</pre> <pre>def capturar_fecha(): bisiesto = False anio = pedir_entero("Ingrese año: ", 2020, 2025) #bisiesto = divisible exacto en 4 y no divisible entre 100 #o es divisible exacto entre 400 if (anio % 4 == 0 and anio % 100 != 0) or (anio % 400 == 0): bisiesto = True mes = pedir_entero("Ingrese mes: ", 1, 12) if mes == 2 and bisiesto: dia = pedir_entero("Ingrese día: ", 1, 29) elif mes == 2 and not bisiesto: dia = pedir_entero("Ingrese día: ", 1, 28) elif mes in [4, 6, 9, 11]: dia = pedir_entero("Ingrese día: ", 1, 30) else: dia = pedir_entero("Ingrese día: ", 1, 31) fecha = (dia, mes, anio) pdb.set_trace() return fecha fecha=capturar_fecha()</pre>
<pre>----Escribir archivo--- Seleccione el archivo al que desea agregar información: (1) catalogo_servicios.txt (2) olticas_del_negocio.txt (3) promociones_especiales.txt (4) bitacora_pedidos.txt ¿Que acción desea realizar? (1) Anexar información (agregar al final) (2) Reescribir archivo (borrar anterior y poner nuevo) > c:\Users\davil\anoddata\local\temp\ipykernel_3892\379635384.py(276)escribir_archivo: 1 'a'</pre> <pre>accion= pedir_entero("Ingrese opción:", 1, 2) if accion == 1: modo = "a" elif accion == 2: modo = "w" pdb.set_trace() nuevo_texto = input("Ingrese la información que desea guardar: ")</pre>	Error: Al probar la función de escribir archivos, los modos de apertura ("a" y "w") estaban invertidos en las condiciones, lo que provocaba que al seleccionar una opción se borrara el archivo anterior por equivocación en lugar de anexar la información. Solución: Se corrigió la asignación de los modos dentro de la estructura condicional para que correspondan correctamente a cada opción elegida. Con el uso del depurador, se comprobó que el programa ahora aplica el modo adecuado ("a" para anexar o "w" para reescribir) sin alterar los datos por error.	<pre>¿Que acción desea realizar? (1) Anexar información (agregar al final) (2) Reescribir archivo (borrar anterior y poner nuevo) > c:\Users\davil\anoddata\local\temp\ipykernel_3892\648819494.py(276)escribir_archivo: *** NameError: name 'accion' is not defined 2 'a'</pre> <pre>accion= pedir_entero("Ingrese opción:", 1, 2) if accion == 1: modo = "w" elif accion == 2: modo = "a" pdb.set_trace() nuevo_texto = input("Ingrese la información que desea guardar: ")</pre>

```
if eleccion == 4:
    print("\n1.- Base de Datos de Clientes ---")
    print(" (1) Ingreso manual (escribir nombre y teléfono libremente)")
    print(" (2) Usar datos del último ticket generado en el programa")
    origen_datos = pedir_entero("Elija una opción", 1, 2)

    if origen_datos == 1:
        nombre_cliente = input("Nombre del cliente: ")
        telefono_cliente = input("Teléfono del cliente: ")
        escribir_archivo(nombre_cliente, telefono_cliente)
    elif origen_datos == 2:
        print("Cerrando sesión. Hasta luego!")
        break

    escribir_archivo()

    #---- Escribir o Editar Archivo ----
    #Seleccione el archivo que desea modificar:
    # [1] catalogo_servicios.txt
    # [2] politicas_del_negocio.txt
    # [3] promociones_especiales.txt
    # [4] bitacora_pedidos.txt
    > c:\users\dayvillanodata\local\temo\linkkernel_34424\1573561867.py(353)main()
5
```

```
Bienvenido al sistema, Maria
RENOVACALZADO
Iniciando sistema...
Cargando módulos... [1/5]
Cargando módulos... [2/5]
Cargando módulos... [3/5]
Cargando módulos... [4/5]
Cargando módulos... [5/5]
¡Carga completada con éxito!

--- Configuración inicial de fecha ---
Fecha: 25/09/2026

--- AVISO DE INACTIVIDAD ---
Han transcurrido 10 minutos sin interacción u opción seleccionada.
> c:\users\dayvillanodata\local\temo\linkkernel_34424\1573561867.py(353)main()
True

continuar = inactividad()
if not continuar:
    print("\nSesión suspendida por inactividad. Regresando al inicio...")
    sesion_activa = False
    break

print("\n=====")
print("\n==Menu Principal==")
print("\n=====")
```

```
for pedido in range(1, c_pedido+1):
    #Ingresar servicio y precio (se pide nombre, ticket y t.
    while True:
        servicio_elegido = pedir_entero("Ingrese Servicio(pedido)/(c_pedido): ", 1, 5)
        servicio_real = servicios_opciones[servicio_elegido-1]
        limite = dic_tiempo_entrega[servicio_real][1]
        if ticket_servicios.count(servicio_real) < limite:
            print("Servicio (servicio_real): ticket_servicios.count(servicio_real) de Máximo (limite)")
            break
        else:
            print("Servicio (servicio_real): Supero el límite.(ticket_servicios.count(servicio_real)/(limite) ")

#Ahorar
ticket_servicios.append(servicio_real)
subtotal += servicio_real(servicio_real)
dic_ticket_servicios[servicio_real] = ticket_servicios.count(servicio_real)

#Calcular tiempo de entrega actualizando t_entrega
cantidad_servicio = ticket_servicios.count(servicio_real)
t_entrega = calcular_tiempo_entrega(servicio_real, t_entrega, cantidad_servicio)
```

Error: No existía la opción 5 en el menú de selección de archivos, y el programa estaba utilizando erróneamente la opción 4 para intentar gestionar la base de datos de clientes.

Solución: Se agregó e integró la opción 5 (Base_datos_clientes.txt) dentro del menú de archivos, ajustando la estructura condicional para que el flujo reconozca correctamente dicha opción y permita registrar los datos del cliente sin confusiones.

Error: La función de inactividad se llamaba en un lugar incorrecto del código, lo que provocaba que ni siquiera se mostrara el menú principal y fallara al intentar detener el programa para regresar al inicio.

Solución: Se reubicó la llamada a la función dentro del flujo principal y se ajustó la condición para que detenga la ejecución correctamente y permita regresar al inicio.

Error: El programa no mostraba los nombres reales de los servicios ni el control correcto de la cantidad sobre el límite, ya que intentaba imprimir directamente el número de opción elegido en lugar del nombre del servicio.

Solución: Se extrajo correctamente el nombre del servicio mediante el diccionario antes de validar los límites (servicio_real = servicios_opciones[servicio_elegido - 1]), logrando que imprima el nombre y la

```
def main():
    #---- Escribir o Editar Archivo ----
    #Seleccione el archivo que desea modificar:
    # [1] catalogo_servicios.txt
    # [2] politicas_del_negocio.txt
    # [3] promociones_especiales.txt
    # [4] bitacora_pedidos.txt
    # [5] Base_datos_clientes.txt

    #--- Base de Datos de Clientes ---
    # (1) Ingreso manual (escribir nombre y teléfono libremente)
    # (2) Usar datos del último ticket generado en el programa
    > c:\users\dayvillanodata\local\temo\linkkernel_37784\1695125667.py(305)escribir_archivo()
5
```

```
while sesion_activa:
    # Encabezado
    print("\n=====")
    print("\n==Menu Principal==")
    print("\n=====")

    # Matriz para imprimir las opciones del menú de forma organizada
    matriz_menu_principal = [
        ["(1) Cotizar Servicio", "(2) Leer Archivos"],
        ["(3) Escribir/anexar documentos", "(4) Cerrar Usuario"]
    ]

    for fila in matriz_menu_principal:
        for columna in fila:
            print(columna)

    print("\n[Iniciando simulación por inactividad]")
    continuar = inactividad()

    if not continuar:
        print("\nSesión suspendida por inactividad. Regresando al inicio...")
        sesion_activa = False
        break
```

```
Terminal Help
EntregaFinalProyec: ¿Qué desea hacer? Elija una opción. (Rango 1-4) (Press 'Enter' to confirm or 'Escape' to cancel)
EntregaFinalProyec(py) > P4 TICKET RENOVACALZADO > P4 NUEVO CODIGO > Q Import pd
Generate + Code + Markdown Interrupt Restart Clear All Outputs Go To Jupyter Variables Out
Fecha: 25/09/2026

=====
==Menu Principal==
=====
(1) Cotizar Servicio
(2) Leer Archivos
(3) Escribir/anexar documentos
(4) Salir

[Iniciando simulación por inactividad]

--- AVISO DE INACTIVIDAD ---
Han transcurrido 10 minutos sin interacción u opción seleccionada.
```

```
while True:
    servicio_elegido = pedir_entero("Ingrese Servicio(pedido)/(c_pedidos): ", 1, 5)
    servicio_real = servicios_opciones[servicio_elegido-1]
    limite = dic_tiempo_entrega[servicio_real][1]
    intentos_actuales = ticket_servicios.count(servicio_real) + 1

    # Validar si no sobrepasa el límite permitido para ese servicio
    if ticket_servicios.count(servicio_real) < limite:
        print(f"Servicio (servicio_real): (intentos_actuales) de Máximo (limite)")
        break
    else:
        print(f"Servicio (servicio_real): Supero el límite.(ticket_servicios.count(servicio_real)/(limite) ")
```

```
=====
=====TICKET=====
=====RENOVACALZADO=====
=====
Nombre cliente: Pedro ignacio
Servicio Costura: 1 de Máximo 3
Servicio Costura: 2 de Máximo 3
Servicio Costura: 3 de Máximo 3
Servicio Tintando: 1 de Máximo 2
Servicio Ajuste: 1 de Máximo 9
```

	cantidad máxima permitida de forma correcta en el ticket.	
<pre> def leer_archivo(self, nombre_archivo): """ Lee el contenido de un archivo de texto. """ try: with open(nombre_archivo, "r") as archivo: contenido = archivo.read() print(contenido) except FileNotFoundError: print(f"El archivo {nombre_archivo} no existe.") except UnicodeDecodeError: print(f"Error de decodificación al leer el archivo {nombre_archivo}.") </pre>	Error: Al intentar leer un archivo de texto, el programa crasheaba por un error de decodificación (UnicodeDecodeError) debido a caracteres incompatibles con la codificación por defecto del sistema operativo. En esta ocupe tanto el pdb como prints de control porque no entendía porque no lo leía aunque el valor sí coincidía. Solución: Se recreó el archivo desde cero con una codificación limpia y estándar, eliminando los caracteres ocultos que causaban el conflicto y permitiendo que el script leyera la bitácora sin problemas.	<pre> --- Leer Archivos --- Archivos disponibles en el sistema: [1] catalogo_servicios.txt [2] politicas_del_negocio.txt [3] promociones_especiales.txt [4] bitacora_pedidos.txt [5] Base_datos_clientes.txt --- Contenido de 'bitacora_pedidos.txt' --- BITACORA DE PEDIDOS. Fecha: 31/12/2025 Cliente: Laura mendoza Total: \$200.00 Fecha: 25/08/2021 Cliente: No se Total: \$2002.00 Fecha: 25/09/2026 Cliente: Alexander uribe Total: \$3410.00 Fecha: 25/09/2026 Cliente: Ophelia Total: \$990.00 Fecha: 28/09/2026 Cliente: Lilliel maria Total: \$558.00 Fecha: 30/12/2026 Cliente: Maria ortega Total: \$1638.00 Fecha: 28/02/2026: Agregando lineas de prueba Fecha: 30/09/2026 Cliente: Pedro ignacio Total: \$1278.00 </pre>
<pre> (4) Salir [Iniciando simulación por inactividad] --- AVISO DE INACTIVIDAD --- Han transcurrido 10 minutos sin interacción u opción seleccionada. [Sesión suspendida por inactividad]. Regresando al inicio... </pre>	Error: Al activarse el aviso por inactividad y elegir que ya no se quería continuar, el programa no regresaba de forma correcta ni cerraba el menú, quedándose atrapado en un bucle dentro del ciclo while. Solución: Se ajustó la estructura de control para que, al suspender la sesión por inactividad, el flujo rompa el ciclo actual, regrese al inicio para pedir de nuevo la identificación del usuario o permita cerrar el programa limpiamente.	<pre> global fecha, formateada_global # Mide el que regresa corriendo sesión o # Después de un periodo de inactividad y selecciona que ya no quiere continuar while True: print("=====") print("UN SISTEMA RENOVACIAZO ") print("=====") # Llamo a la función login(), aquí elige: identificarse o salir de todo el programa if not login(): return # Inicia la pantalla de carga una vez identificado pantalla_carga() </pre> <pre> def login(): print("\nIngrese usuario (o escriba 'Salir' para cerrar el sistema). ") usuario = input("").capitalize() if usuario == "Salir": print("Cerrando el sistema. ¡Hasta luego!") return False print(f"\nBienvenido al sistema, {usuario} ") return True def pantalla_carga(): </pre>